ISTQB CERTIFIED TESTER FOUNDATION LEVEL V4.0

Complete Study Notes

Exam-focused notes for the six examinable chapters of the CTFL v4.0 syllabus: fundamentals of testing, testing throughout the SDLC, static testing, test analysis and design techniques, managing the test activities, and tool support - with keywords, comparison tables and memory aids.

Chapters 1 - 6 · Syllabus v4.0 (2023)

KEYWORDS · LEARNING OBJECTIVES · EXAM FOCUS

Table of Contents

How to Use These Notes 2

Chapter 1 - Fundamentals of Testing (8 questions) 2

1.1 Test Objectives and Testing vs. Debugging . . . 2

1.2 Why Testing Is Necessary . . . 3

1.3 The Seven Testing Principles - memorize all seven . . . 3

1.4 Test Process, Testware and Roles . . . 4

1.5 Skills, Whole-Team Approach and Independence . . . 5

Chapter 2 - Testing Throughout the Software Development Lifecycle (7 questions) 5

2.1 Good Practices, Test-First Approaches, DevOps and Shift-Left . . . 6

2.2 The Five Test Levels and Four Test Types . . . 7

2.3 Maintenance Testing . . . 7

Chapter 3 - Static Testing (4 questions) 8

3.1 The Review Process - five activities in order . . . 9

3.2 Roles and Responsibilities in Reviews . . . 9

3.3 The Four Review Types and Success Factors . . . 9

Chapter 4 - Test Analysis and Design (13 questions) 10

4.1 Equivalence Partitioning (EP) . . . 10

4.2 Boundary Value Analysis (BVA) . . . 11

4.3 Decision Table Testing . . . 11

4.4 State Transition Testing . . . 11

4.5 White-Box Techniques: Statement and Branch Coverage . . . 12

4.6 Experience-Based Techniques . . . 12

4.7 Collaboration-Based Approaches: User Stories, Acceptance Criteria, ATDD . . . 13

Chapter 5 - Managing the Test Activities (6 questions) 14

5.1 Test Planning and the Tester's Contribution . . . 14

5.2 Entry Criteria, Exit Criteria, DoR and DoD . . . 14

5.3 The Four Estimation Techniques - K3 territory . . . 15

5.4 Test Case Prioritization, Test Pyramid, Testing Quadrants . . . 15

5.5 Risk Management . . . 16

5.6 Monitoring, Reports, Configuration and Defect Management . . . 17

Chapter 6 - Test Tools (2 questions) 18

Final Word - How the Chapters Connect 18

How to Use These Notes

These notes compress the six examinable chapters of the official CTFL v4.0 syllabus into exam-focused prose. Every chapter opens with its keyword box - the K1 terms the exam can test directly - and follows the syllabus structure so you can always cross-reference the official document. Sentences that sound definitional are definitional on purpose: the exam tests the exact distinctions the syllabus draws, so when these notes say confirmation testing confirms an original fix, that precise wording is what distinguishes the right answer from the tempting one.

Work chapter by chapter with the study plan in Document 1. For each section: read it once, close the notes, and write down what you remember; compare and repair; then answer the matching chapter drill in Document 3 with the notes still closed. Tables in these notes are your fastest review artifacts the night before the exam, together with the two-page cheat sheet in Document 4. Chapter headings here carry the official question weight - eight, seven, four, thirteen, six, and two questions respectively - so you always know where the next point lives.

Chapter 1 - Fundamentals of Testing (8 questions)

coverage • debugging • defect • error • failure • quality • quality assurance • root cause • test analysis • test basis • test case • test completion • test condition • test control • test data • test design • test execution • test implementation • test monitoring • test object • test objective • test planning • test procedure • test result • testing • testware • validation • verification

Testing is a set of activities conducted to facilitate discovery and evaluation of defects in test artifacts. The artifacts being tested are called test objects. A common misconception is that testing consists only of executing tests; in reality testing also includes planning, analysis, design, implementation, reporting, and evaluation of work products such as requirements and user stories. Testing may be dynamic (executing the software) or static (reviews and static analysis, which do not require execution). Testing involves verification, which checks whether the system meets specified requirements, and validation, which checks whether the system meets users' needs in its operational environment.

1.1 Test Objectives and Testing vs. Debugging

Typical test objectives include evaluating work products such as requirements, designs, and code; triggering failures and finding defects; ensuring required coverage of a test object; reducing the risk of inadequate software quality; verifying whether specified requirements have been fulfilled; verifying compliance with contractual, legal, and regulatory requirements; providing information to stakeholders so they can make informed decisions; building confidence in the quality of the test object; and validating whether the test object is complete and works as the stakeholders expect. Objectives vary with the context: the work product being tested, the test level, the risks, the software development lifecycle, and business factors such as time to market.

Testing and debugging are separate activities. Dynamic testing can trigger failures caused by defects; debugging then locates the cause of the failure (the defect), analyzes it, and removes it. The typical debugging process has three steps: reproduction of the failure, diagnosis (finding the root cause), and fixing the cause. Confirmation testing afterwards checks whether the fix actually resolved the problem, and regression testing checks that the fix did not break anything else. Static testing finds defects directly, without any failure, so debugging after static testing only involves removing the defect - there is no failure to reproduce or diagnose.

1.2 Why Testing Is Necessary

Testing contributes to success in several ways. It provides a cost-effective means of detecting defects that are then removed by debugging, so it indirectly contributes to higher quality. It directly evaluates the quality of a test object at various stages of the SDLC, and those measures feed management decisions such as the release decision. It gives users an indirect representation on the development project, because testers ensure the understanding of user needs is considered throughout the lifecycle. Testing may also be required to satisfy contractual, legal, or regulatory requirements.

> **Testing vs. Quality Assurance - a favorite exam distinction**
>
> Quality control (QC) is product-oriented and corrective: it focuses on activities that support achieving appropriate quality levels, and testing is its major form. Quality assurance (QA) is process-oriented and preventive: it focuses on implementing and improving processes, and it applies to both development and testing. Test results are used in QC to fix defects and in QA as feedback on how well the processes perform.

Human beings make errors (mistakes), which produce defects (faults, bugs), which in turn may result in failures. Errors come from time pressure, complexity, process or infrastructure problems, fatigue, or lack of training. A defect in code, when executed, may cause the system to do something it should not, or fail to do what it should: a failure. Some defects always cause failures when executed, others only in special circumstances, and some may never cause a failure at all. Failures can also be caused by environmental conditions such as radiation or electromagnetic fields, without any human error. A root cause is the fundamental reason a problem occurred; root cause analysis is triggered when a failure occurs or a defect is identified, and addressing the root cause prevents or reduces similar future failures.

1.3 The Seven Testing Principles - memorize all seven

#	Principle	One-line meaning
1	Testing shows the presence, not the absence of defects	Testing can prove defects exist, never that none remain
2	Exhaustive testing is impossible	Use techniques, prioritization and risk analysis to focus effort
3	Early testing saves time and money	Start static and dynamic testing as early as possible

#	Principle	One-line meaning
4	Defects cluster together	A small number of components holds most defects (Pareto); feeds risk-based testing
5	Tests wear out	Repeated tests stop finding new bugs; modify and add tests (except useful automated regression)
6	Testing is context dependent	No single universal approach; do it differently per context
7	Absence-of-defects fallacy	A fully verified system can still fail users - validation is also needed

These seven principles appear in the exam as direct K1 recall and as scenario judgments. The traps are predictable: a question claiming testing can prove correctness contradicts principle 1; a question claiming every input combination can be tested contradicts principle 2; a question stating that a defect-free system guarantees business success contradicts principle 7, because verification alone is not enough without validation.

1.4 Test Process, Testware and Roles

The test process consists of the following main activity groups, which are often implemented iteratively or in parallel and must be tailored to the project: test planning, test monitoring and control, test analysis, test design, test implementation, test execution, and test completion. Test analysis answers the question what to test by analyzing the test basis to define and prioritize test conditions with their risks. Test design answers how to test by elaborating test conditions into test cases and other testware, identifying coverage items, and defining test data and environment requirements. Test implementation creates or acquires the testware needed for execution, organizes test cases into test procedures and suites, prioritizes them into an execution schedule, and builds the test environment. Test execution runs the tests according to the schedule and logs results, and test completion consolidates data, testware, and experience at milestones such as the end of a test level or project.

Testware is the collection of artifacts produced during testing: test plans, test conditions, test cases, test procedures, test data, test logs, defect reports, and more. Testware has a useful lifetime beyond a single project iteration and should be maintained under configuration management. Traceability between the test basis (requirements, user stories, designs) and the testware supports coverage evaluation, impact analysis of changes, and audit requirements. Good traceability lets you answer three exam-style questions: what coverage did we achieve, which tests are affected by a changed requirement, and which requirement does this defect relate to.

Two testing roles frame the process. The test manager (or test lead) plans, monitors, and controls the testing overall, and in sequential lifecycles often holds this role explicitly. Testers analyze, design, implement, and execute tests, log results, and report defects. In Agile projects the test management tasks are often distributed across the whole team, and testing itself is done by testers working alongside developers and business representatives.

1.5 Skills, Whole-Team Approach and Independence

Testing is an intellectual activity, and the syllabus lists the generic skills testers need: testing knowledge (to increase effectiveness), thoroughness, care, attention to detail and methodical working; good communication skills, active listening, and being a team player; analytical thinking, critical thinking, creativity, curiosity, and technical knowledge; and domain knowledge, because testers using the same language as business stakeholders improve communication and defect reporting.

The whole team approach leverages the collective knowledge and skills of the team: developers and business representatives share quality responsibility with testers, the team leverages each member's skill set for the project's benefit, and quality becomes everyone's responsibility. It is a defining characteristic of Agile testing and reduces the communication distance between testing and development. Its limit is also an exam favorite: losing independence, which is why independent testing still matters.

Independence of testing comes in levels, from no independence (the developer tests their own code) through independent developers or testers within the team, to an independent test team or organizational unit, up to independent external organizations. Higher independence brings an outside view that finds defects the author is blind to, challenges assumptions in specifications, and can report issues more objectively. Its drawbacks are isolation from the development team, longer feedback loops, and possible loss of developer responsibility for quality. The right level is a context decision, and in Agile the whole-team approach combined with some independent test specialists is common.

Chapter 2 - Testing Throughout the Software Development Lifecycle (7 questions)

acceptance testing • black-box testing • component integration testing • component testing • confirmation testing • functional testing • integration testing • maintenance testing • non-functional testing • regression testing • shift-left • system integration testing • system testing • test level • test object • test type • white-box testing

A software development lifecycle (SDLC) model is an abstract representation of how the development process unfolds. Sequential models such as waterfall and the V-model deliver code late, so dynamic testing starts late while requirement reviews and test analysis happen early. Iterative and incremental models deliver a working increment every iteration, so both static and dynamic testing run at all levels each cycle, and frequent delivery demands fast feedback plus extensive regression testing. Agile development assumes change everywhere, favoring lightweight documentation, heavy test automation to keep regression cheap, and manual testing that leans on experience-based techniques. The chosen SDLC directly impacts the scope and timing of test activities, the level of detail of test documentation, the choice of test techniques and approach, the extent of test automation, and the tester's role and responsibilities.

2.1 Good Practices, Test-First Approaches, DevOps and Shift-Left

Four good testing practices apply regardless of the SDLC: for every development activity there is a corresponding test activity; different test levels have specific and different objectives so testing is comprehensive without redundancy; test analysis and design for a given level begin during the corresponding development phase, honoring the principle of early testing; and testers review work products as soon as drafts exist, supporting shift-left defect detection.

TDD, ATDD, and BDD are test-first approaches in which tests direct development and are written before the code they verify. TDD directs coding through test cases: write the test, write the code to satisfy it, then refactor both. ATDD derives tests from acceptance criteria as part of system design, before the application part is developed. BDD expresses the desired behavior in simple natural language, usually the Given/When/Then format, which stakeholders understand, and those test cases are automatically translated into executable tests. In all three, tests typically persist as automated tests protecting future refactoring.

DevOps creates synergy by getting development, including testing, and operations to work toward common goals, promoting team autonomy, fast feedback, integrated toolchains, and CI/CD practices. Testing benefits include fast feedback on code quality, a strengthened shift-left approach because developers submit component tests and static analysis with their code, stable test environments through automated pipelines, more attention to non-functional characteristics, less repetitive manual testing, and minimized regression risk thanks to large automated regression suites. The challenges are defining the pipeline, introducing and maintaining CI/CD tooling, and the resource cost of automation - and even with high automation, manual testing from the user's perspective is still needed.

Shift-left means moving testing earlier in the SDLC, not abandoning late testing. Its concrete practices are reviewing specifications from a testing perspective, writing test cases before the code exists and running code in a test harness during implementation, using CI (and better CD) for fast feedback with automated component tests, completing static analysis of source code before or alongside dynamic testing, and starting non-functional testing at component level where possible. It may add training, effort, and cost early, in exchange for savings later - and stakeholders must buy into it for it to work.

> **Retrospectives and process improvement**
>
> Retrospectives are held at the end of a project, iteration, or release milestone. Participants - not only testers - discuss what was successful and should be kept, what was not successful and could be improved, and how to incorporate improvements going forward. Results are recorded and normally become part of the test completion report. For testing they bring increased effectiveness and efficiency, higher testware quality, team bonding and learning, improved test basis quality, and better development-testing cooperation.

2.2 The Five Test Levels and Four Test Types

Level	Focus	Typical owner / notes
Component testing (unit)	Components in isolation; needs test harnesses or unit frameworks	Developers, in their development environment
Component integration testing	Interfaces and interactions between components	Depends on integration strategy: bottom-up, top-down, big-bang
System testing	Overall behavior and capabilities of the whole system, end-to-end	Independent test team often; based on system specifications
System integration testing	Interfaces of the system with other systems and external services	Needs environments similar to production
Acceptance testing	Validation and deployment readiness; user business needs	Ideally by intended users; forms: UAT, operational, contractual and regulatory, alpha, beta

Test levels are distinguished by five attributes that prevent overlap: test object, test objectives, test basis, defects and failures expected, and approach and responsibilities. In sequential models the exit criteria of one level feed the entry criteria of the next; in iterative models levels may overlap in time. Test types are groups of activities related to quality characteristics, and any of them can be applied at every level. Functional testing evaluates what the system should do - completeness, correctness, and appropriateness of functions. Non-functional testing evaluates how well the system behaves, using the ISO/IEC 25010 characteristics: performance efficiency, compatibility, usability, reliability, security, maintainability, and portability. Black-box testing is specification-based, checking behavior against documentation external to the test object. White-box testing is structure-based, deriving tests from the implementation's internal structure to cover it to an acceptable level.

> **Confirmation vs. regression - the most confused pair in the exam**
>
> Confirmation testing confirms that an original defect has been successfully fixed - re-run the failed tests, add new tests for the change, or at minimum reproduce the original failure. Regression testing confirms that no adverse consequences were caused by a change, possibly far beyond the changed component. Both are needed on all test levels wherever defects are fixed or changes are made. Run impact analysis first to size the regression suite, automate it early, and remember its case count grows with every iteration.

2.3 Maintenance Testing

Maintenance testing tests changes to a system that is already in operation. Maintenance can be corrective, adaptive to environment changes, or aimed at improving performance or maintainability, and it covers both planned releases and unplanned hot fixes. Impact analysis may be performed before the change is made to decide whether it is worth making, based on consequences elsewhere in the system. Testing the change includes both evaluating the success of its implementation and checking for regressions in the parts that

remained unchanged, which usually means most of the system. The scope of maintenance testing depends on the degree of risk of the change, the size of the existing system, and the size of the change itself.

The triggers for maintenance and its testing fall into three groups. Modifications include planned enhancements, corrective changes, and hot fixes. Upgrades or migrations of the operational environment, for example moving platform, require testing in and of the new environment, including data conversion when data is migrated from another application. Retirement happens when the application reaches end of life and can require testing of data archiving and of restore and retrieval procedures while data must remain accessible during the retention period.

Chapter 3 - Static Testing (4 questions)

anomaly • dynamic testing • formal review • informal review • inspection • review • static analysis • static testing • technical review • walkthrough

In static testing the software under test does not need to be executed. Work products such as code, process specifications, and architecture documents are evaluated through manual examination (reviews) or with tool support (static analysis). Objectives include improving quality, detecting defects, and assessing characteristics such as readability, completeness, correctness, testability, and consistency. Static testing can serve both verification and validation. Almost any work product that can be read and understood can be reviewed - requirements, source code, test plans, test cases, backlog items, contracts, models. Static analysis, by contrast, needs a structure to check against: code with formal syntax, models, or formatted text. Work products inappropriate for static testing include things hard for humans to interpret and that tools should not analyze, such as third-party executable code for legal reasons.

Static analysis identifies problems before dynamic testing, often with less effort since no test cases are required, and is typically built into CI frameworks. Beyond finding specific code defects it also evaluates maintainability and security; spelling checkers and readability tools count as static analysis too. The value of static testing is that it finds defects in the earliest phases, honors the early-testing principle, catches defect types dynamic testing cannot (unreachable code, design patterns not implemented as intended, defects in non-executable products), improves communication among stakeholders, and usually lowers total project cost despite the cost of the reviews themselves.

Static and dynamic testing complement each other. Static testing finds defects directly, while dynamic testing triggers failures from which defects are then analyzed. Static testing more easily detects defects on rarely executed or hard-to-reach code paths, works on non-executable work products, and measures quality characteristics independent of execution such as maintainability; dynamic testing measures what depends on executing code, such as performance efficiency. Typical defects that are cheaper to find statically: requirements inconsistencies, ambiguities, contradictions, omissions, inaccuracies and duplications; design defects like inefficient database structures; coding defects such as undefined or undeclared variables, unreachable or duplicated code, excessive complexity; deviations from standards; incorrect interface specifications; security vulnerabilities like buffer overflows; and gaps in test basis coverage.

3.1 The Review Process - five activities in order

Reviews follow a generic process defined in ISO/IEC 20246 that is tailored up or down in formality. The five activities are: planning, where the scope is defined - purpose, work product, quality characteristics, focus areas, exit criteria, supporting information, effort, and timeframes; review initiation, which ensures everyone and everything is prepared, every participant has access, understands their role, and receives what they need; individual review, where each reviewer applies one or more review techniques such as checklist-based or scenario-based reviewing and logs anomalies, recommendations, and questions; communication and analysis, typically in a review meeting, where anomalies are analyzed - they are not necessarily defects - and status, ownership, and required actions are decided for each; and fixing and reporting, where defect reports are created for corrective action, exit criteria are checked, the work product is accepted, and results are reported. Large work products are reviewed in chunks, invoking the process several times.

3.2 Roles and Responsibilities in Reviews

Role	Responsibility
Manager	Decides what is to be reviewed; provides time, staff, and resources
Author	Creates and fixes the work product under review
Moderator (facilitator)	Ensures effective review meetings: mediation, time management, safe environment
Scribe (recorder)	Collates anomalies from reviewers; records decisions and new anomalies
Reviewer	Performs the review; may be a project member, subject matter expert, or stakeholder
Review leader	Overall responsibility: who is involved, when and where the review happens

In inspections - the most formal review type - the author cannot act as the review leader or the scribe. Remember that pairing for trap questions: author writes and fixes; moderator runs the meeting; scribe records; review leader organizes; manager funds and decides.

3.3 The Four Review Types and Success Factors

Type	Formality	Key characteristics and objectives
Informal review	Lowest	No defined process, no documented output; main objective is detecting anomalies
Walkthrough	Low-medium	Led by the AUTHOR; quality evaluation, confidence building, educating reviewers, consensus, new ideas, author improvement, anomaly detection; individual review beforehand optional
Technical review	Medium-high	Technically qualified reviewers, led by a MODERATOR; consensus and decisions on technical problems, plus anomaly detection and quality evaluation

Type	Formality	Key characteristics and objectives
Inspection	Highest	Follows the complete generic process; main objective is finding the MAXIMUM number of anomalies; metrics collected to improve the process; author cannot be review leader or scribe

The required formality depends on the SDLC, process maturity, the criticality and complexity of the work product, legal or regulatory requirements, and the need for an audit trail. The same work product can be reviewed with different types at different times. Success factors include defining clear objectives and measurable exit criteria - participant evaluation must never be an objective; choosing the appropriate review type; reviewing small chunks so concentration holds; feeding results back to stakeholders and authors; giving participants enough preparation time; having management support; making reviews part of the culture; training everyone for their role; and facilitating meetings well. Early and frequent stakeholder feedback is itself a quality weapon: it prevents misunderstandings about requirements, ensures requirement changes are understood and implemented earlier, and keeps the team focused on the features delivering the most value.

Chapter 4 - Test Analysis and Design (13 questions)

acceptance criteria • **acceptance test-driven development** • **black-box test technique** • **boundary value analysis** • **branch coverage** • **checklist-based testing** • **collaboration-based test approach** • **coverage** • **coverage item** • **decision table testing** • **equivalence partitioning** • **error guessing** • **experience-based test technique** • **exploratory testing** • **state transition testing** • **statement coverage** • **test technique** • **white-box test technique**

Test techniques support test analysis (what to test) and test design (how to test), helping you develop a relatively small but sufficient set of test cases in a systematic way. Black-box techniques (specification-based) analyze the specified behavior without reference to internal structure, so tests stay valid when the implementation changes but behavior does not. White-box techniques (structure-based) analyze the internal structure and processing, so they can only be designed after design or implementation. Experience-based techniques exploit tester knowledge and skill, and detect defects the other two families miss - they are complementary, not alternatives.

4.1 Equivalence Partitioning (EP)

EP divides data into partitions where all elements are expected to be processed the same way: if one value from a partition reveals a defect, any other value from the same partition should reveal it too, so one test per partition suffices. Partitions can be built for inputs, outputs, configuration items, internal values, time-related values, and interface parameters, and they may be continuous or discrete, ordered or unordered, finite or infinite. Partitions must not overlap and must be non-empty. A partition containing valid values is a valid partition; one containing invalid values is an invalid partition, and both must be tested. The coverage items are the partitions themselves; 100 percent coverage means every identified partition is exercised at least once, and coverage equals partitions exercised divided by total partitions. With multiple input parameters, Each Choice coverage requires each partition of each set to be covered at least once, ignoring combinations.

4.2 Boundary Value Analysis (BVA)

BVA exercises the boundaries of ordered partitions, because developers are statistically more likely to make errors exactly at those boundaries. Typical defects are boundaries shifted above or below their intended position, or omitted entirely. In 2-value BVA each boundary has two coverage items: the boundary value and its closest neighbor belonging to the adjacent partition. In 3-value BVA each boundary has three coverage items: the boundary value and both its neighbors, so some coverage items are not boundary values. 3-value BVA is more rigorous: if the decision if (x <= 10) is wrongly implemented as if (x == 10), the 2-value tests (10, 11) miss the defect, but x = 9 from 3-value BVA is likely to catch it. Coverage is the number of coverage items exercised divided by the total number of coverage items, in both variants.

> **Worked micro-example - practice this pattern**
>
> For a valid range 1-100: valid partition [1..100], invalid partitions x < 1 and x > 100. Boundaries are 1 and 100. 2-value BVA tests: 0, 1, 100, 101. 3-value BVA adds the second neighbors: 2 and 99, giving 0, 1, 2, 99, 100, 101.

4.3 Decision Table Testing

Decision tables test requirements that specify how different combinations of conditions produce different outcomes, making them an effective way to record complex business rules. Conditions and actions form the rows; each column is a decision rule defining a unique condition combination with its actions. In limited-entry tables all condition and action values are Boolean; in extended-entry tables they may take ranges, partitions, or discrete values. Notation: T means the condition is satisfied, F that it is not, a dash that it is irrelevant to the outcome, and N/A that it is infeasible for that rule; X marks an action that should occur, blank one that should not. A full table has a column for every combination; it can be simplified by deleting infeasible columns and minimized by merging columns whose conditions do not affect the outcome. The coverage items are the columns with feasible combinations; 100 percent coverage means every feasible column is exercised. Be ready for the exponential warning: rules grow as 2 to the power of the number of conditions, so many conditions demand minimization or a risk-based selection.

4.4 State Transition Testing

A state transition diagram models behavior through possible states and valid transitions, where a transition is initiated by an event, may be qualified by a guard condition, and may trigger an action; the common label syntax is event [guard condition] / action. A state table is the equivalent matrix: rows are states, columns are events, cells hold the target state and actions, and empty cells explicitly mark invalid transitions - which the diagram does not show. A test case is a sequence of events producing a sequence of state changes, and one test case usually covers several transitions.

Coverage criterion	Coverage items	Notes
All states coverage	The states	Weakest: usually achievable without exercising all transitions
Valid transitions (0-switch)	Single valid transitions	Most widely used; full valid-transitions coverage guarantees full all-states coverage
All transitions	All transitions in the state table (valid + invalid)	Includes attempting invalid transitions; test ONE invalid transition per case to avoid fault masking; minimum for mission and safety-critical software

Chain them: full all-transitions coverage guarantees full valid-transitions coverage and full all-states coverage; full valid-transitions coverage guarantees full all-states coverage. Fault masking - one defect preventing the detection of another - is the reason invalid transitions are tested one per test case.

4.5 White-Box Techniques: Statement and Branch Coverage

In statement testing the coverage items are executable statements; coverage is statements exercised divided by total executable statements. One hundred percent statement coverage ensures every executable statement ran at least once, so a defective statement was executed - but execution does not guarantee failure detection, since defects can be data dependent, and it does not ensure all decision logic was tested. In branch testing the coverage items are branches, the transfers of control between nodes of the control flow graph; conditional branches correspond to the outcomes of if decisions, switch/case outcomes, and loop exit or continue decisions. Branch coverage subsumes statement coverage: any test set achieving 100 percent branch coverage also achieves 100 percent statement coverage, but not vice versa.

The shared strength of all white-box techniques is that the entire implementation is taken into account, enabling defect detection even when specifications are vague, outdated, or incomplete. The corresponding weakness is defects of omission: if the software fails to implement a requirement, white-box testing may not notice, because there is nothing in the structure to miss. White-box ideas also serve static testing - dry runs and reviews of code or pseudocode modeled as control flow graphs. And while black-box testing alone gives no measure of actual code coverage, white-box measures provide an objective number and the information needed to add tests and grow confidence.

4.6 Experience-Based Techniques

Error guessing anticipates the occurrence of errors, defects, and failures based on the tester's knowledge: how the application behaved in the past, the errors developers tend to make, and the failures seen in similar applications. Errors cluster around input, output, logic, computation, interfaces, and data. Fault attacks are its methodical implementation: build or acquire a list of possible errors, defects, and failures, then design tests that expose them.

Exploratory testing designs, executes, and evaluates tests simultaneously while the tester learns about the test object, then explores deeper with focused tests and creates tests for untested areas. It is often structured

through session-based testing: a defined time-box, a test charter with the test objectives guiding the session, and a debriefing with stakeholders afterwards, with test session sheets documenting steps and discoveries. It is the technique of choice with few or inadequate specifications, or under significant time pressure, and it complements formal techniques; effectiveness grows with the tester's experience, domain knowledge, curiosity, creativity, and analytical skill, and it can incorporate other techniques such as equivalence partitioning.

Checklist-based testing designs, implements, and executes tests to cover test conditions from a checklist built from experience, user priorities, and knowledge of why software fails. Items are usually phrased as questions, each checkable separately and directly, and may reference requirements, interface properties, or quality characteristics. Checklists should exclude items checkable automatically, items better handled as entry or exit criteria, and items too general. They lose effectiveness as developers learn to avoid old mistakes, so update them regularly from defect analysis - but keep them short. High-level checklists give guidelines and consistency but less repeatability, with potentially greater coverage.

4.7 Collaboration-Based Approaches: User Stories, Acceptance Criteria, ATDD

A user story represents a feature valuable to a user or purchaser, and has three critical aspects, the 3 Cs: Card, the medium describing the story; Conversation, explaining how the software will be used, documented or verbal; and Confirmation, the acceptance criteria. The most common format is As a [role], I want [goal], so that [business value], followed by the acceptance criteria. Good stories are INVEST: Independent, Negotiable, Valuable, Estimable, Small, Testable - and if a stakeholder cannot say how to test a story, the story may be unclear or not valuable to them. Collaborative writing uses brainstorming and mind mapping and forces three perspectives: business, development, and testing.

Acceptance criteria are the conditions an implementation of the story must meet to be accepted, and they act as the test conditions the tests should exercise. They define the scope of the story, build stakeholder consensus, describe positive and negative scenarios, serve as the basis for acceptance testing, and allow accurate planning and estimation. The two most common formats are scenario-oriented, such as the Given/When/Then format used in BDD, and rule-oriented, such as a bullet-point verification list or an input-output mapping table. Any custom format is acceptable as long as criteria are well-defined and unambiguous.

ATDD is the test-first application of this collaboration: test cases are created before the story is implemented, by team members with customer, developer, and tester perspectives. The first step is a specification workshop where the story and its acceptance criteria are analyzed, discussed, and written, resolving incompleteness and ambiguity. Test cases are then created by the team or by the tester, based on the acceptance criteria, and act as examples of how the software should work. Typically the first tests are positive, then negative tests follow, and finally non-functional characteristics are covered. Test cases must cover all characteristics of the story without going beyond it, and no two test cases should describe the same characteristics. When captured in an automation-friendly format they become executable requirements.

Chapter 5 - Managing the Test Activities (6 questions)

defect management • defect report • entry criteria • exit criteria • product risk • project risk • risk • risk analysis • risk assessment • risk control • risk identification • risk level • risk management • risk mitigation • risk monitoring • risk-based testing • test approach • test completion report • test control • test monitoring • test plan • test planning • test progress report • test pyramid • testing quadrants

5.1 Test Planning and the Tester's Contribution

A test plan describes the test project's objectives, resources, and processes: it documents the means and schedule for achieving test objectives, helps ensure activities meet established criteria, serves as communication with team members and stakeholders, and demonstrates adherence to organizational test policy and strategy - or explains any deviation. Typical content: context of testing including scope, objectives, constraints, and test basis; assumptions and constraints; stakeholders; communication forms and frequency; the risk register; the test approach including levels, types, techniques, deliverables, entry and exit criteria, independence, metrics, test data and environment requirements; and budget and schedule. In iterative SDLCs testers add value in release planning - writing testable user stories with acceptance criteria, participating in project and quality risk analysis, estimating test effort, determining the test approach - and in iteration planning - detailed risk analysis of user stories, testability checks, breaking stories into tasks including testing tasks, estimating testing tasks, and identifying functional and non-functional aspects to test.

5.2 Entry Criteria, Exit Criteria, DoR and DoD

Entry criteria define the preconditions for starting an activity; if they are unmet the activity becomes more difficult, slower, costlier, and riskier. Exit criteria define what must be achieved to declare the activity completed. Both should be defined for each test level and differ with test objectives. Typical entry criteria: availability of resources such as people, tools, environments, test data, budget, and time; availability of testware such as test basis, testable requirements, user stories, and test cases; and an initial quality level of the test object, for example all smoke tests passing. Typical exit criteria: measures of thoroughness such as achieved coverage, number of unresolved defects, defect density, and failed test counts; and completion criteria such as planned tests executed, static testing performed, all found defects reported, and regression tests automated. Running out of time or budget can count as valid exit criteria when stakeholders accept the risk. In Agile, exit criteria are called Definition of Done - the team's objective metrics for a releasable item - and the entry criteria for a user story are the Definition of Ready.

5.3 The Four Estimation Techniques - K3 territory

Technique	Type	How it works
Estimation based on ratios	Metrics-based	Collect figures from previous projects, derive standard ratios; e.g. dev-to-test 3:2 with 600 dev person-days gives 400 test person-days
Extrapolation	Metrics-based	Measure early in the current project and extrapolate with a mathematical model; suits iterative SDLCs, e.g. average of last three iterations
Wideband Delphi	Expert-based, iterative	Experts estimate in isolation, discuss out-of-range deviations, re-estimate in isolation until consensus; Planning Poker is the Agile variant using cards
Three-point estimation	Expert-based	Optimistic a, most likely m, pessimistic b; E = (a + 4m + b) / 6 and SD = (b - a) / 6; e.g. a=6, m=9, b=18 gives E=10, SD=2

> **Formula to burn in**
>
> Expect at least one three-point calculation. E = (a + 4m + b) / 6; SD = (b - a) / 6. With a = 6, m = 9, b = 18 person-hours: E = (6 + 36 + 18) / 6 = 10, SD = (18 - 6) / 6 = 2, reported as 10 plus or minus 2 person-hours. Always state the unit in your answer choice.

5.4 Test Case Prioritization, Test Pyramid, Testing Quadrants

Test suites are arranged in a test execution schedule defining the order of execution. Risk-based prioritization runs test cases covering the most important risks first. Coverage-based prioritization runs the cases achieving the highest coverage first, with the additional-coverage variant picking each next case for the highest additional coverage. Requirements-based prioritization follows stakeholder priorities of the requirements traced to the cases. Two constraints override priority order: dependencies - a high-priority case dependent on a lower-priority case forces the lower one first - and resource availability such as tools, environments, or people available only in specific time windows.

The test pyramid shows that different tests have different granularity and supports test automation and effort allocation. Higher layers mean lower granularity, lower isolation, and longer execution time; bottom-layer tests are small, isolated, fast, and numerous, while the top layer holds few, complex, slow, end-to-end tests. Layer names vary - Cohn's unit, service, UI, or component, integration, end-to-end. The testing quadrants, from Brian Marick, group test levels with test types and techniques along two axes: business facing vs. technology facing, and supporting the team vs. critiquing the product.

Quadrant	Facing	Purpose	Typical tests	Automation
Q1	Technology	Support the team	Component and component integration tests	Should be automated, part of CI

Quadrant	Facing	Purpose	Typical tests	Automation
Q2	Business	Support the team	Functional tests, examples, user story tests, UX prototypes, API testing, simulations	Manual or automated
Q3	Business	Critique the product	Exploratory testing, usability testing, UAT	User-oriented, often manual
Q4	Technology	Critique the product	Smoke tests and non-functional tests except usability	Often automated

5.5 Risk Management

Risk is a potential event, hazard, threat, or situation whose occurrence causes an adverse effect. Two factors characterize it: risk likelihood, the probability of occurrence greater than zero and less than one, and risk impact, the harm of the occurrence. Together they express the risk level, and the higher the level the more important the treatment. The main risk management activities are risk analysis - risk identification plus risk assessment - and risk control - risk mitigation plus risk monitoring. The test approach that selects, prioritizes, and manages activities based on this cycle is called risk-based testing.

Project risks concern management and control of the project: organizational issues like delays, inaccurate estimates, cost-cutting; people issues like insufficient skills, conflicts, staff shortage; technical issues like scope creep and poor tool support; and supplier issues like third-party failure. They threaten schedule, budget, or scope. Product risks concern product quality characteristics: missing or wrong functionality, incorrect calculations, runtime errors, poor architecture, inefficient algorithms, inadequate response time, poor user experience, security vulnerabilities. Their consequences range from user dissatisfaction and lost revenue or reputation to damage to third parties, high maintenance costs, criminal penalties, and in extreme cases injury or death.

Product risk analysis consists of risk identification - generating a comprehensive risk list using brainstorming, workshops, interviews, or cause-effect diagrams - and risk assessment: categorizing risks, determining likelihood, impact, and level, prioritizing, and proposing handling. Assessment may be quantitative, multiplying likelihood by impact, or qualitative, using a risk matrix. Its results determine the scope of testing, the test levels and types, the techniques and coverage, the effort per task, the test priorities, and whether non-test activities could reduce risk. Risk control then applies the response options: mitigation by testing, risk acceptance, risk transfer, or contingency plan. Mitigation by testing includes selecting testers with the right experience for the risk type, applying an appropriate level of test independence, conducting reviews and static analysis, applying appropriate techniques and coverage, applying the test types addressing the affected quality characteristics, and performing dynamic testing including regression.

5.6 Monitoring, Reports, Configuration and Defect Management

Test monitoring gathers information about testing progress against the plan and the exit criteria. Test control uses that information to issue corrective directives: reprioritizing tests when a risk becomes an issue, re-evaluating entry or exit criteria after rework, adjusting schedules for delayed environments, and adding resources. Test completion collects data from completed activities at milestones - a finished test level, iteration, or project. Common metrics: project progress such as task completion and resource usage; test progress such as cases run, passed, failed and execution time; product quality such as availability, response time, and mean time to failure; defect metrics such as counts, priorities, density, and detection percentage; risk metrics like residual risk level; coverage metrics for requirements and code; and cost metrics.

Test progress reports are produced regularly during monitoring and control and include: the test period, progress against schedule with deviations, impediments and workarounds, the metrics, new and changed risks, and testing planned for the next period. A test completion report is prepared once, at completion, and includes: test summary, evaluation of testing and product quality against the plan's objectives and exit criteria, deviations from the plan, impediments and workarounds, metrics drawn from progress reports, unmitigated risks and unfixed defects, and lessons learned. Audiences differ - team-level reporting is frequent and informal, project completion reporting is templated and singular - and communication channels range from verbal and dashboards to email, documentation, and formal reports, chosen per stakeholder need.

Configuration management provides the discipline for identifying, controlling, and tracking work products - test plans, strategies, conditions, cases, scripts, results, logs, reports - as configuration items. Approved items become baselines changeable only through formal change control; new baselines record changes; and reverting to a previous baseline reproduces previous test results. CM ensures all items including test items are uniquely identified, version controlled, tracked, and related for traceability, and that all documentation and software items are unambiguously referenced in test documentation.

Defect management runs anomalies from discovery to closure: logging, analyzing and classifying, deciding the response - fix or keep - and closing. Reported anomalies may turn out to be false positives or change requests; that is resolved in the process. Defect reports exist to give enough information to resolve the issue, to track work product quality, and to feed process improvement. A defect report from dynamic testing typically includes: unique identifier; title summarizing the anomaly; date, organization, and author with role; identification of test object and test environment; context such as the test case and activity; description of the failure with steps to reproduce, logs, screenshots; expected and actual results; severity; priority to fix; status such as open, deferred, duplicate, closed, rejected; and references such as to the test case.

Chapter 6 - Test Tools (2 questions)

test automation

Test tools support and facilitate many test activities, and the exam expects you to match tool categories to activities. Management tools increase test process efficiency by facilitating management of the SDLC, requirements, tests, defects, and configuration. Static testing tools support reviews and static analysis. Test design and implementation tools facilitate generating test cases, test data, and test procedures. Test execution and coverage tools facilitate automated execution and coverage measurement. Non-functional testing tools perform testing that is difficult or impossible manually. DevOps tools support the delivery pipeline, workflow tracking, automated builds, and CI/CD. Collaboration tools facilitate communication, and tools for scalability and deployment standardization include virtual machines and containerization. Any tool assisting testing counts - even a spreadsheet, in the context of testing.

Acquiring a tool guarantees nothing: each tool demands effort for introduction, maintenance, and training before benefits become real and lasting. Potential benefits: time saved by reducing repetitive manual work such as regression execution, re-entering test data, comparing expected with actual results, and checking coding standards; prevention of simple human errors through greater consistency and repeatability; more objective assessment such as coverage measures too complex for humans; easier access to information for test management and reporting; reduced execution times giving earlier defect detection, faster feedback, and faster time to market; and more tester time for designing deeper, more effective tests.

Potential risks mirror the benefits: unrealistic expectations about tool functionality and ease of use; underestimating the time, cost, and effort to introduce the tool, maintain scripts, and change the manual process; using a tool where manual testing is more appropriate; over-reliance that ignores human critical thinking; vendor dependency when the vendor fails, retires, or sells the tool; open-source abandonment or churn; incompatibility with the development platform; and unsuitable tools that violate regulatory or safety requirements.

Final Word - How the Chapters Connect

The exam loves boundary questions where two chapters meet. Chapter 1 defines the defect chain that Chapter 3's static testing attacks without execution. Chapter 2's test levels are where Chapter 4's techniques are applied and where Chapter 5's quadrants live. Chapter 5's risk level feeds the prioritization strategies and the maintenance scope decisions of Chapter 2. When you answer a question, place it in this map first: most wrong options are true sentences from the wrong neighborhood.

Review rhythm for the last days: reread every table in these notes, then every keyword box, then the cheat sheet. Anything you cannot reproduce from memory goes back into the error notebook you kept while drilling. Walk into the exam knowing the seven principles, the five levels, the four types, the four review types, the three state-transition coverage criteria, the four estimation techniques, and the two axes of the testing quadrants - those lists alone answer a double-digit number of the forty questions.